

DATA 266 — Homework 1

Autoregressive models, diabetes neural nets, and CUDA matmul

Sneha Singh

SID4 = 0670 | SEED = 670 | SLICE = 670 | HP_ID = 4 | CLS_A = 0 | CLS_B = 5

HP_ID 4: same network [64, 32] and lr 0.001, 15 epochs instead of 30.

Git repo: https://github.com/snehas-SJSU/Data266-0670/tree/main/HW_1

1. What are autoregressive models?

An autoregressive model predicts the next value using previous values of the same series. That is it. If I want tomorrow's temperature, I use the last few days. If I want the next word, I use the words so far.

AR(1) means I only use the last step. If I use more past steps it is AR(p). You can also write it as a probability chain: $p(x) = p(x_1) p(x_2 | x_1) p(x_3 | x_1, x_2) \dots$ so each piece is predicted from what already came. A neural net can do those steps too (pNeural on the slide).

Real examples: weather, stock prices (AR / ARIMA), more than one series together (VAR), and GPT-style models that pick the next token.

2. Diabetes dataset — neural networks

I used diabetes.csv. 759 rows, 8 features, and Outcome is 0 or 1. No missing values. More 1s than 0s (496 vs 263). The numbers already look scaled around -1 to 1. I still scaled again, but only using the training split.

I split 70/15/15 with random_state=670. That gave train 531, val 114, test 114. I used this same split for every model. Baseline is hidden layers [64, 32], learning rate 0.001, 30 epochs. For HP_ID 4 I kept the same layers and lr and just trained 15 epochs.

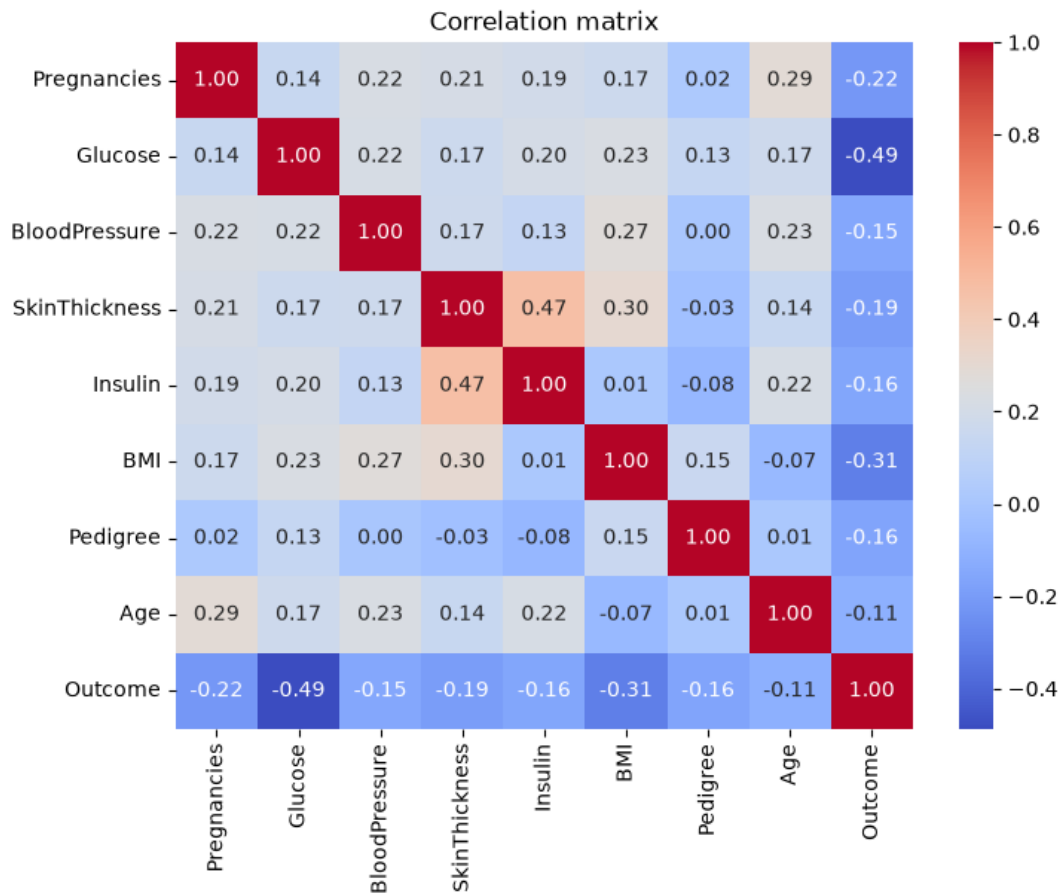


Figure 1. Correlation matrix of the diabetes features and Outcome.

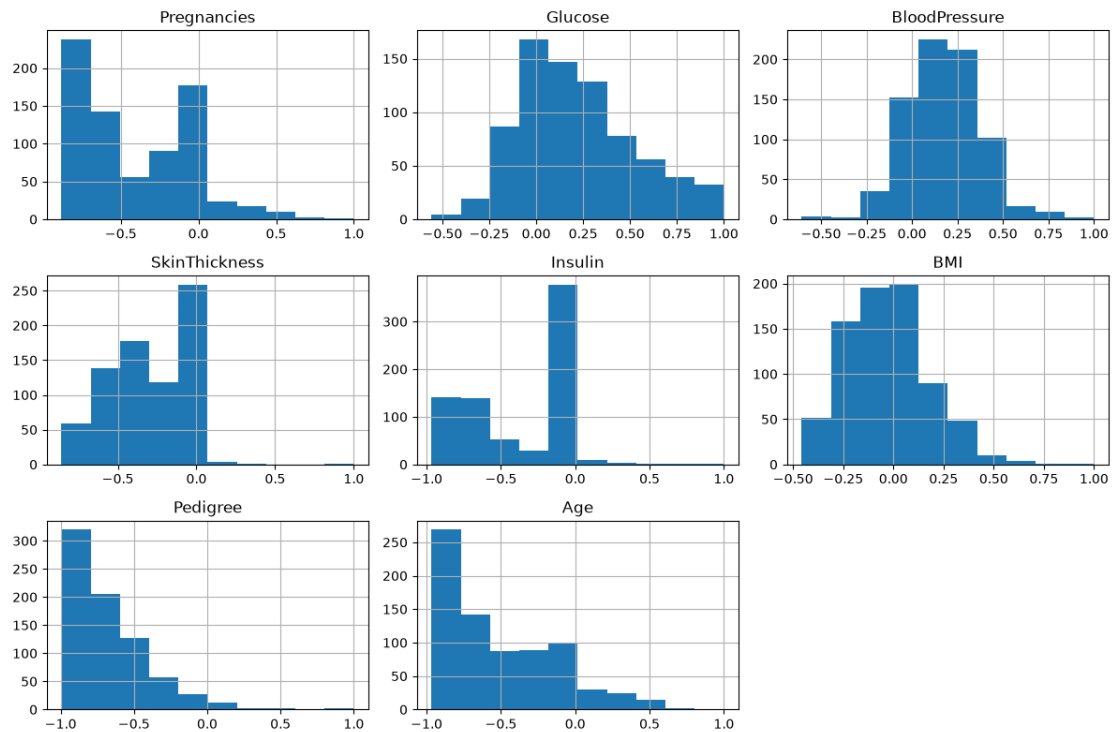


Figure 2. Feature distributions (histograms).

2.1 Seed 670 loss curves

I plotted train and val loss for seed 670 on one figure per framework. Blue/orange are the 30 epoch baseline. Green/red stop at 15 epochs.

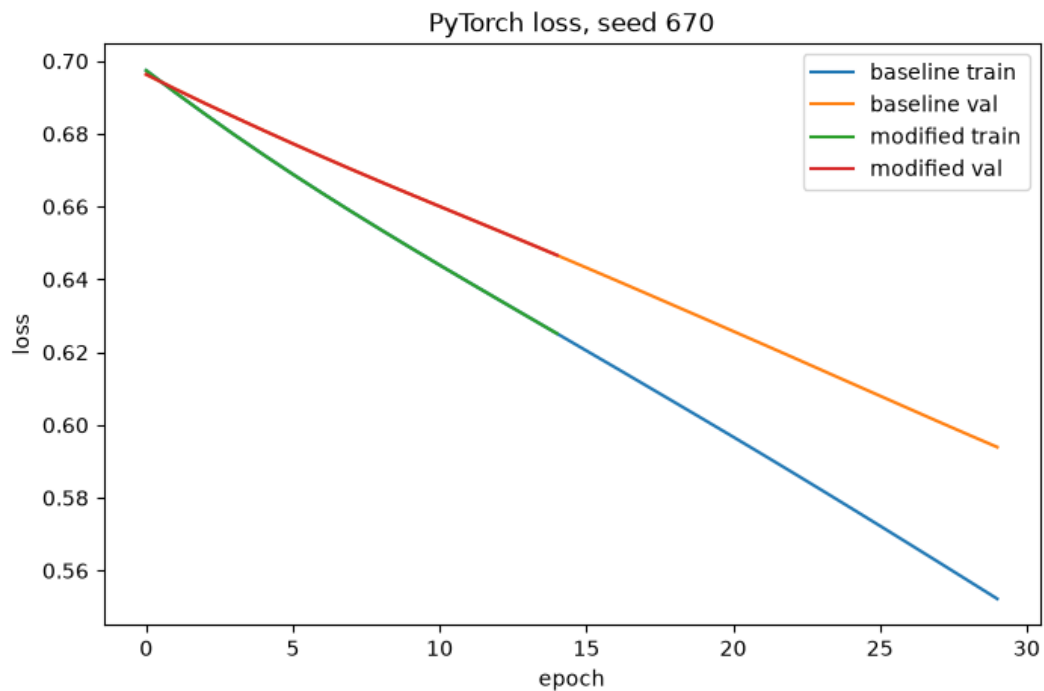


Figure 3. PyTorch train/val loss, seed 670. Test acc: baseline 0.70, modified 0.62.

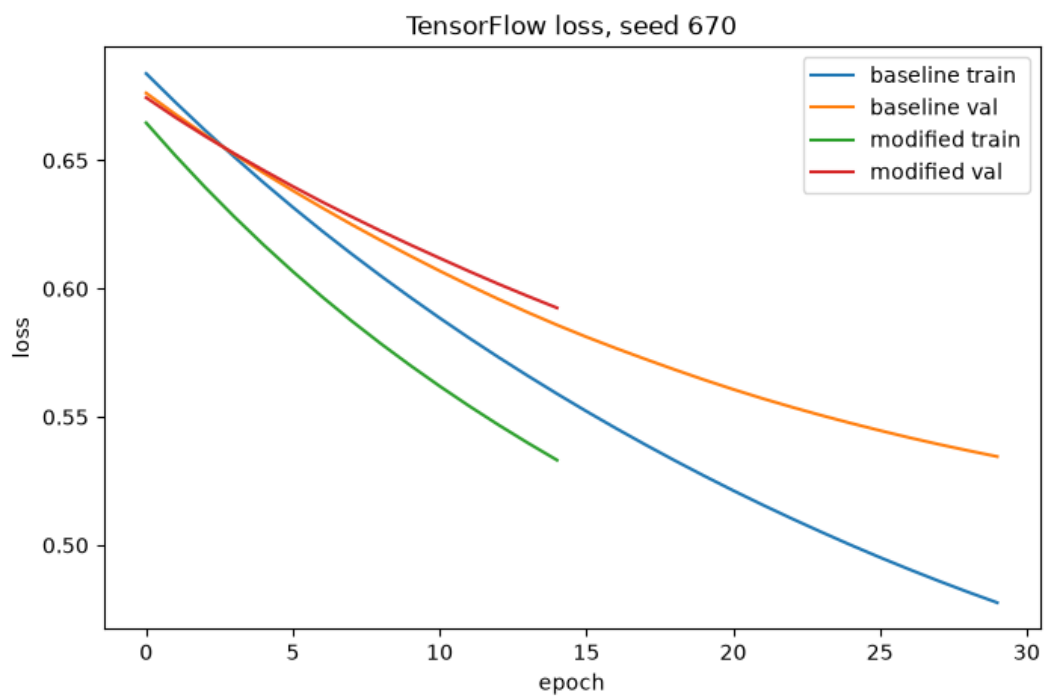


Figure 4. TensorFlow train/val loss, seed 670. Test acc: baseline 0.72, modified 0.65.

2.2 Test accuracy (3 training seeds)

I retrained with seeds 670, 671 and 672. Same split. Metric is test accuracy. I set `tf.random.set_seed` each time, but TensorFlow still jumped: seed 670 baseline was 0.72 in the plot cell and 0.66 in the 3-seed cell. So I am using the 3-seed table, not the plot-cell TF numbers.

Framework	Model	Config	Mean	Std
PyTorch	Baseline	[64,32], lr 0.001, 30 ep	0.6696	0.0252
PyTorch	Modified (HP_ID 4)	[64,32], lr 0.001, 15 ep	0.5994	0.0180
TensorFlow	Baseline	[64,32], lr 0.001, 30 ep	0.6784	0.0149
TensorFlow	Modified (HP_ID 4)	[64,32], lr 0.001, 15 ep	0.6374	0.0083

Framework	Model	seed 670	seed 671	seed 672
PyTorch	Baseline	0.7018	0.6404	0.6667
PyTorch	Modified	0.6228	0.5965	0.5789
TensorFlow	Baseline	0.6579	0.6930	0.6842
TensorFlow	Modified	0.6316	0.6491	0.6316

2.3 Conclusion

HP_ID 4 only changed how long I train: 15 epochs vs 30. Same [64, 32] and same lr 0.001. On seed 670 the shorter run did worse, PyTorch 0.70 to 0.62 and TensorFlow 0.72 to 0.65. On the loss plots, train and val are still going down at epoch 15, and the 30 epoch run keeps going lower. Val loss is not going back up, so I don't think it overfit. I would call the modified model underfitting. It just stopped too early. The 3-seed table looks like that too. Baseline is higher in both PyTorch and TensorFlow.

3. CUDA matrix multiplication

I wrote $C = A * B$ in CUDA. Each block is 16 x 16 threads. One thread does one output:

`row = blockIdx.y * blockDim.y + threadIdx.y`

`col = blockIdx.x * blockDim.x + threadIdx.x`

That thread writes $C[\text{row}, \text{col}]$. If N is not a multiple of 16 the extra threads just return. I ran this on Colab Tesla T4. I did a warmup first, then one timed run for each size (CPU, kernel, and the copies). I did not run it several times and average. N=4096 on CPU took about 13 minutes. `nsys` was not on Colab so I used `nvprof`. `nvprof` says `gpu_matmul` is 88.8% of GPU time, copy to the GPU 6.6%, copy back 4.6%.

N	CPU (ms)	GPU kernel (ms)	H2D+D2H (ms)	Speedup
256	25.308	0.146	0.437	43.38
1024	3291.187	9.187	4.756	236.05
4096	763322.870	362.831	72.603	1753.02

Speedup = CPU / (kernel + H2D+D2H). N=256 max |CPU - GPU| = 1.79e-07.

3.1 Crossover

The smallest size I measured where GPU end-to-end is faster is N=256: CPU 25.308 ms vs kernel + copies 0.146 + 0.437 = 0.583 ms (about 43x). I did not run anything smaller than 256, so that is the first point in my table, not an exact search for the switch.

Crossover is not at size 0 because the GPU time I report is not just the multiply. It also includes copying the two input matrices to the GPU and the result back to the CPU, plus starting the kernel. At N=256 that copy time is 0.437 ms and the kernel is only 0.146 ms. CPU is 25 ms, so GPU is faster at this size, but a tiny matrix would mostly pay that copy cost and could lose.